NumPy

Dr. Sanjit Kumar Saha

Gisma
University
of Applied
Sciences

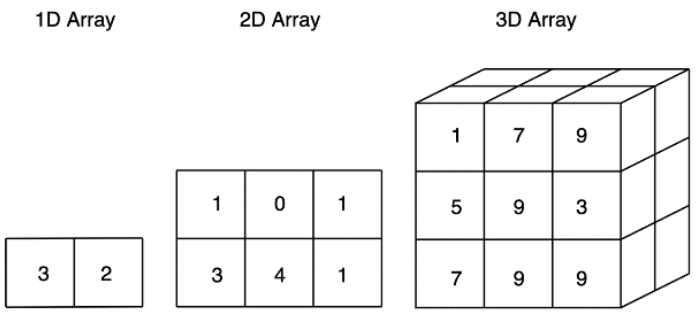
Gisma University of Applied Sciences

Data Arrays

[VanderPlas 2016]

https://towardsdatascience.com/numpy-array-cookbook-generating-and-manipulating-arrays-in-python-2195c3988b09

- Datasets are heterogenous
  - They can come from a wide range of sources and a wide range of formats
- However, it will help us to think of all data fundamentally as arrays of numbers
  - Images can be thought of as simply two-dimensional arrays of numbers representing pixel brightness across the area
  - Sound clips can be thought of as one-dimensional arrays of intensity versus time
  - Text can be converted in various ways into numerical representations
  - Efficient storage and manipulation of numerical arrays is absolutely fundamental
    - No matter what the data are, the first step in making them analyzable will be to transform them into arrays of numbers



NumPy

[VanderPlas 2016]

- NumPy is a Python library.
- NumPy is used for working with arrays.
- NumPy is short for "Numerical Python".

NumPy

Why Use NumPy?

[VanderPlas 2016]

- In Python we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ***ndarray*** very easy.
- Arrays are very frequently used in data science, where speed and resources are very important.

Installation

[VanderPlas 2016]

- To begin using NumPy, you need to install it first. This can be done using the following pip command:

  pip install numpy

- Once installed, import the library with the alias np

  import numpy as np

Creating NumPy Arrays

[VanderPlas 2016]

- Using ***np.array***: Use *np.array()* when you want to convert Python lists into NumPy arrays.

```
import numpy as np

a1 = np.array([1, 2, 3])     # 1D array
a2 = np.array([[1, 2], [3, 4]]) # 2D array
a3 = np.array([[[1, 2], [3, 4]],     # 3D
array
                [[5, 6], [7, 8]]])
```

Numpy Functions

[VanderPlas 2016]

- NumPy provides quick utility functions for creating arrays filled with zeros, ones, or ranges:

```
import numpy as np

a0 = np.zeros((3, 3))
a1  = np.ones((2, 2))
ar  = np.arange(0, 10, 2)
```

NumPy Array Indexing

[VanderPlas 2016]

- Advanced indexing in NumPy uses arrays of integers or boolean masks to extract complex patterns of elements, enabling non-contiguous and condition-based selection.

```
import numpy as np

a1 = np.array([10, 20, 30, 40, 50])
print(a1[2])      # single element
print(a1[-1])     # last element

a2 = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
print(a2[1, 0])   # row 1, column 0
```

Slicing

[VanderPlas 2016]

- Slicing follows the same indexing rules as lists, but extends them to multiple dimensions, allowing to select rows, columns, or sub-arrays efficiently.

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a[1:4])      # row slice
print(a[:, 1])     # all rows, column 1
```

Array Slicing: Accessing Subarrays

[VanderPlas 2016]

- *x[start:stop:step]*
  - We can also access subarrays with the slice notation, marked by the colon (:) character
  - The NumPy slicing syntax follows that of the standard Python list
  - They default to the values start=0, stop=size of dimension, step=1
- Examples for 1D arrays
  - *x[:5] # first five elements*
  - *x[::2] # every other element*
  - *x[::-1] # all elements, reversed*
  - *x[5::-2] # reversed every other from index 5*
- Examples for multidimensional arrays
  - *x2[:2, :3] # two rows, three columns*
  - *x2[:3, ::2] # all rows, every other column*
  - *x2[::-1, ::-1]*

Advanced Indexing

[VanderPlas 2016]

- Advanced Indexing in NumPy provides more flexible ways to access and manipulate array elements.

```
import numpy as np

a = np.array([10, 20, 30, 40, 50, 60])
idx = np.array([1, 3, 5])

print(a[idx])        # integer indexing

cond = a > 30
print(a[cond])       # boolean indexing
```

Basic Arithmetic Operations

[VanderPlas 2016]

- Element-wise operations in NumPy allow to perform mathematical operations on each element of an array individually, without the need for explicit loops.
- We can perform arithmetic operations like addition, subtraction, multiplication and division directly on NumPy arrays.

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])


print(x + y)    # add
print(x - y)    # subtract
print(x * y)    # multiply
print(x / y)    # divide
```

Unary Operation

[VanderPlas 2016]

- Unary operations in NumPy apply a single-operand transformation-such as negation, absolute value, or trigonometric evaluation-across entire arrays efficiently without the need for multiple arrays (as in binary operations).

```
import numpy as np

a = np.array([-3, -1, 0, 1, 3]) # array with
both positive and negative values

# Applying a unary operation: absolute value
print(np.absolute(a))
```

Binary Operators

[VanderPlas 2016]

- Binary Operations apply to the array elementwise and a new array is created. We can use all basic arithmetic operators like +, -, /, etc. Operators like +=, -=, *= modify the existing array in-place and = operator simply assigns a new reference to a variable, it does not modify the original array.

```
import numpy as np

# Two example arrays
a1 = np.array([1, 2, 3])
a2 = np.array([4, 5, 6])

# Applying a binary operation: addition
res = np.add(a1, a2)
print(res)
```

Mathematical Functions

[VanderPlas 2016]

- NumPy provides familiar mathematical functions such as sin, cos, exp, etc. These functions also operate elementwise on an array, producing an array as output.

```
import numpy as np

# create an array of sine values
a = np.array([0, np.pi/2, np.pi])
print(np.sin(a))

# exponential values
b = np.array([0, 1, 2, 3])
print(np.exp(b))
print(np.sqrt(b))

# square root of array values
print (np.sqrt(a))
```

Sorting Arrays

[VanderPlas 2016]

- We can use a simple np.sort() method for sorting Python NumPy arrays.
- Note: Strings with prefix b'' indicate byte strings (fixed-length string in NumPy). S10 means each string can store up to 10 bytes.

```
import numpy as np

dtype = [('name', 'S10'), ('year', int),
('cgpa', float)]
vals  = [('Hrithik', 2009, 8.5),
         ('Ajay',    2008, 8.7),
         ('Pankaj',  2008, 7.9),
         ('Aakash',  2009, 9.0)]

a = np.array(vals, dtype=dtype)

print(np.sort(a, order='name'))
print(np.sort(a, order=['year', 'cgpa']))
```

Array Attributes

[VanderPlas 2016]

- Each array has attributes
  - *ndim*
    - The number of dimensions
  - *shape*
    - The size of each dimension
  - *size*
    - The total size of the array
  - *dtype*
    - The data type of the array
  - *itemsize*
    - The size (in bytes) of each array element
  - *nbytes*
    - The total size (in bytes) of the array

```
np.random.seed(0)
x3 = np.random.randint(10, size=(3, 4, 5))
print("x3 ndim: ", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size: ", x3.size)
print("dtype:", x3.dtype)
print("itemsize:", x3.itemsize, "bytes")
print("nbytes:", x3.nbytes, "bytes")
```

NumPy Array Shape

[VanderPlas 2016]

- NumPy arrays have an attribute called shape that returns a tuple with each index having the number of corresponding elements.
- Print the shape of a 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

print(arr.shape)
```

- The example above returns (2, 4), which means that the array has 2 dimensions, where the first dimension has 2 elements and the second has 4.

NumPy Array Reshaping

[VanderPlas 2016]

- Reshaping means changing the shape of an array.
- The shape of an array is the number of elements in each dimension.
- By reshaping we can add or remove dimensions or change number of elements in each dimension.

Reshape From 1-D to 2-D

[VanderPlas 2016]

- Convert the following 1-D array with 12 elements into a 2-D array.
- The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])

newarr = arr.reshape(4, 3)

print(newarr)
```

Random Numbers in NumPy

[VanderPlas 2016]

- NumPy offers the random module to work with random numbers.
- Generate a random integer from 0 to 100:

```
from numpy import random

x = random.randint(100)
```

Generate Random Array

[VanderPlas 2016]

- The randint() method takes a size parameter where you can specify the shape of an array.
- Generate a 1-D array containing 5 random integers from 0 to 100:

```
from numpy import random

x=random.randint(100, size=(5))
```

- Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100:

```
from numpy import random

x = random.randint(100, size=(3, 5))
```

NumPy ufuncs

[VanderPlas 2016]

- ufuncs stands for "Universal Functions" and they are NumPy functions that operate on the ndarray object.
- ufuncs are used to implement vectorization in NumPy which is way faster than iterating over elements.
- They also provide broadcasting and additional methods like reduce, accumulate etc. that are very helpful for computation.
- ufuncs also take additional arguments, like:
  - *where* boolean array or condition defining where the operations should take place.
  - *dtype* defining the return type of elements.
  - *out* output array where the return value should be copied.

Add the Elements of Two Lists

[VanderPlas 2016]

- NumPy has a ufunc for this, called add(x, y) that will produce the same result.

```
import numpy as np

x = [1, 2, 3, 4]
y = [4, 5, 6, 7]
z = np.add(x, y)
```

Array Concatenation

[VanderPlas 2016]

- *np.concatenate* takes a tuple or list of arrays as its first argument
  - *x = np.array([1, 2, 3])*
  - *y = np.array([3, 2, 1])*
  - *np.concatenate([x, y])*
- *np.concatenate* can also be used for two-dimensional arrays
  - *grid = np.array([[1, 2, 3], [4, 5, 6]])*
  - *np.concatenate([grid, grid]) # concatenate along the first axis*
  - *np.concatenate([grid, grid], axis=1) # concatenate along the second axis*

Array Splitting

[VanderPlas 2016]

- The opposite of concatenation is splitting, which is implemented by the functions *np.split*, *np.hsplit*, and *np.vsplit*
  - *x = [1, 2, 3, 99, 99, 3, 2, 1]*
  - *x1, x2, x3 = np.split(x, [3, 5])*
- For each of these, we can pass a list of indices giving the split points
  - Notice that *N* split points lead to *N + 1* subarrays
- The related functions *np.hsplit* and *np.vsplit* are similar
  - *grid = np.arange(16).reshape((4, 4))*
  - *upper, lower = np.vsplit(grid, [2])*
  - *left, right = np.hsplit(grid, [2])*
- Similarly, *np.dsplit* will split arrays along the third axis

Aggregations: Min, Max, and Everything in Between

[VanderPlas 2016]

- NumPy has fast built-in aggregation functions for working on arrays
  - *L = np.random.random(100)*
  - *np.sum(L)*
  - A shorter syntax is to use methods of the array object itself
    - *L.sum()*
- Be careful that the sum function and the np.sum function are not identical
  - Which can sometimes lead to confusion
  - Whenever possible, make sure that you are using the NumPy version of these aggregates when operating on NumPy arrays
  - NumPy's corresponding functions have similar syntax, and again operate much more quickly than their similar built-in functions

Function Name	NaN-safe Version	Description
`np.sum`	`np.nansum`	Compute sum of elements
`np.prod`	`np.nanprod`	Compute product of elements
`np.mean`	`np.nanmean`	Compute median of elements
`np.std`	`np.nanstd`	Compute standard deviation
`np.var`	`np.nanvar`	Compute variance
`np.min`	`np.nanmin`	Find minimum value
`np.max`	`np.nanmax`	Find maximum value
`np.argmin`	`np.nanargmin`	Find index of minimum value
`np.argmax`	`np.nanargmax`	Find index of maximum value
`np.median`	`np.nanmedian`	Compute median of elements
`np.percentile`	`np.nanpercentile`	Compute rank-based statistics of elements
`np.any`	N/A	Evaluate whether any elements are true
`np.all`	N/A	Evaluate whether all elements are true

Multidimensional Aggregates

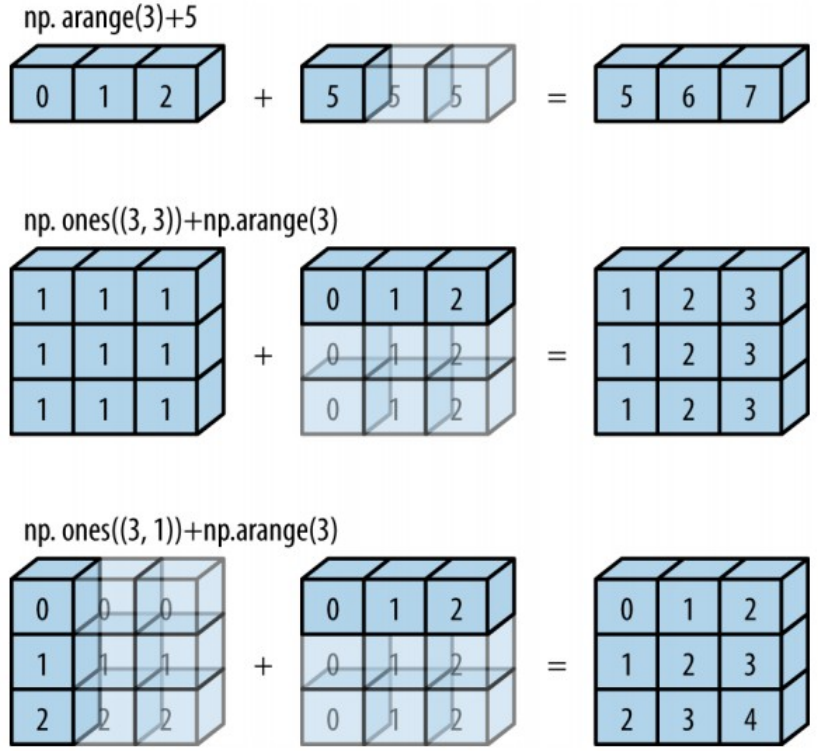
[VanderPlas 2016]

- By default, each NumPy aggregation function will return the aggregate over the entire array
  - *M = np.random.random((3, 4))*
  - *M.sum()*
- Aggregation functions take an additional argument specifying the axis
  - Along which the aggregate is computed
    - *M.min(axis=0) # minimum value within each column*
    - *M.max(axis=1) # maximum value within each row*
  - The axis keyword specifies the dimension of the array that will be collapsed, rather than the dimension that will be returned

Computation on Arrays: Broadcasting

[VanderPlas 2016]

- Broadcasting is simply a set of rules for applying binary UFuncs on arrays of different sizes
  - For example, *np.arange(3) + 5*
    - We can think of this as an operation that stretches or duplicates the value *5* into the *array [5, 5, 5]*, and adds the results
  - We can similarly extend this to arrays of higher dimension
- The light boxes represent the broadcasted values
  - This extra memory is not actually allocated in the course of the operation, but it can be useful conceptually to imagine that it is



Rules of Broadcasting

[VanderPlas 2016]

- Rule 1
  - If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side
    - *M.shape: (2, 3) -> (2, 3)*
    - *a.shape: (3,) -> (1, 3)*
- Rule 2
  - If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape
    - *M.shape: (2, 3) -> (2, 3)*
    - *a.shape: (1, 3) -> (2, 3)*
- Rule 3
  - If in any dimension the sizes disagree and neither is equal to 1, an error is raised
    - M.shape: (3, 2)
    - a.shape: (3, 3)